Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №8
з дисципліни «Технології та методика програмування вбудованих систем»

«Дослідження моделі модуля АЦП та цифрового вольтметра»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 06.04.2026

КИЇВ – 2026

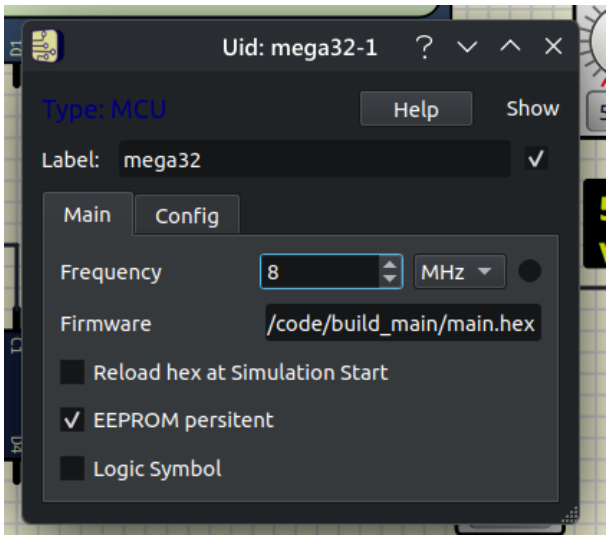
Тема. Моделювання модуля АЦП мікроконтролерів сім'ї AVR. Моделювання цифрового вольтметра.

Мета. Користуючись пакетом SimulIDE дослідити роботу модуля АЦП та дослідити моделювання цифрового вольтметра.

Завдання. Створити модель модуля АЦП та модель цифрового вольтметра в пакеті SimulIDE. Розробити схеми алгоритмів роботи цих моделей та робочі програми. Створити відповідні hex-файли та підключити їх до мікроконтролера. Запустити моделі та виконати їх дослідження згідно методичних вказівок. Зробити відповідні висновки.

Хід роботи.

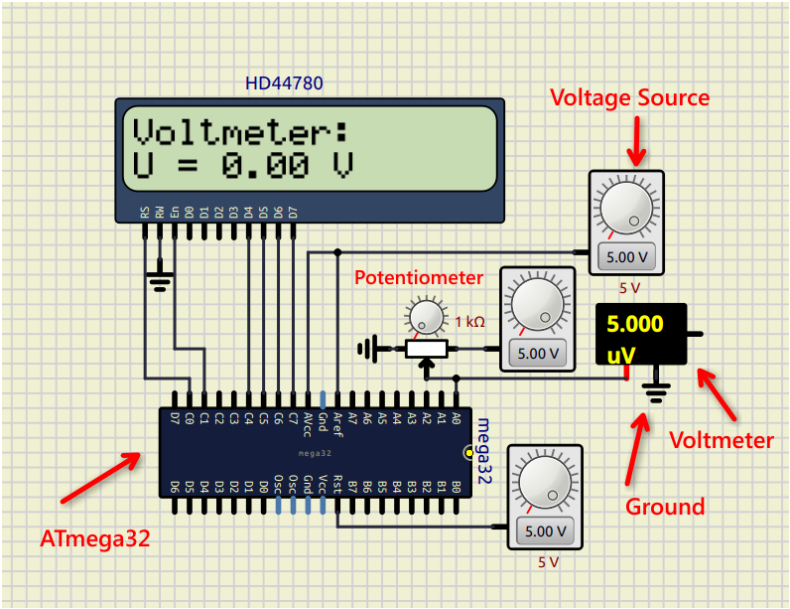
Виконання восьмої лабораторної роботи було присвячено дослідженню архітектури та принципів роботи вбудованого модуля аналого-цифрового перетворювача (АЦП) мікроконтролерів сімейства AVR на прикладі створення моделі цифрового вольтметра. Основу обчислювального ядра склав мікроконтролер ATmega32, тактова частота якого була встановлена на 8 МГц.



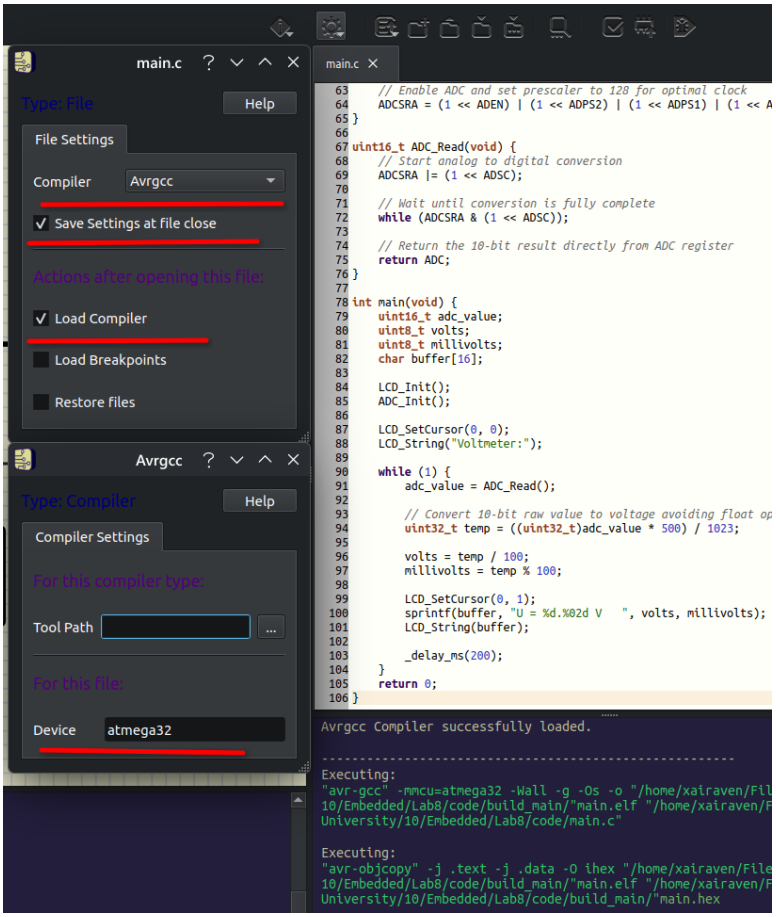
Для виведення результатів вимірювання було використано символьний рідкокристалічний дисплей HD44780, підключений до порту C мікроконтролера в оптимізованому 4-бітному режимі керування.

В якості джерела змінного аналогового сигналу було використано потенціометр (змінний резистор), підключений за схемою дільника напруги між шиною живлення 5 В та загальним дротом (`GND`). Центральний вивід потенціометра (повзунок) був з'єднаний з піном PA0 мікроконтролера, який апаратно відповідає нульовому каналу мультиплексора АЦП (`ADC0`). Для забезпечення коректної роботи аналого-цифрового перетворювача, його ізольовані лінії живлення (`AVcc`) та

джерела опорної напруги (`Aref`) були підключені до стабільного джерела 5 В. Паралельно до виходу потенціометра було підключено віртуальний вольтметр для візуального контролю реальної напруги та порівняння її з результатами програмної обробки мікроконтролером.



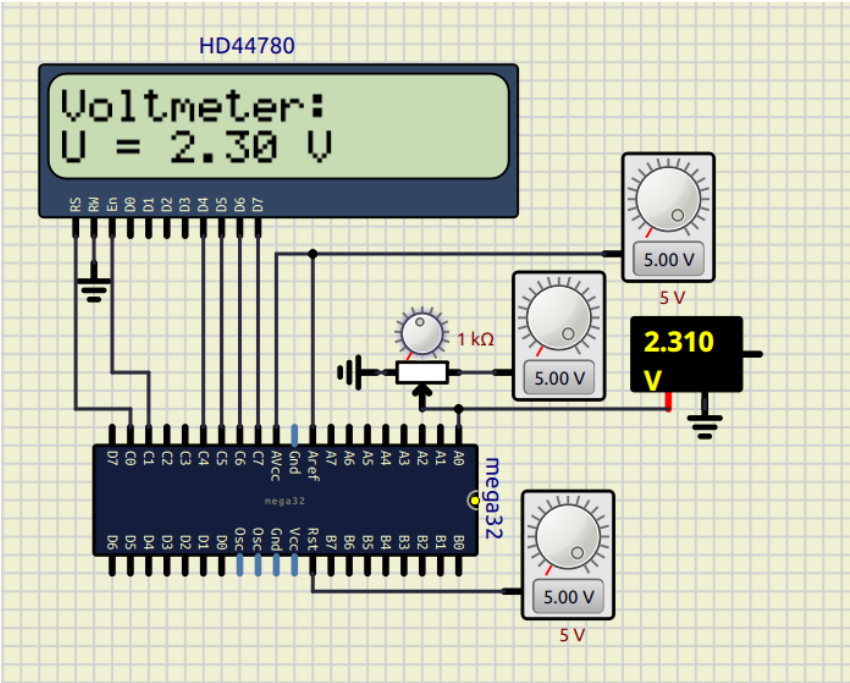
Програмна частина моделі була написана мовою C з використанням компілятора `avr-gcc`.



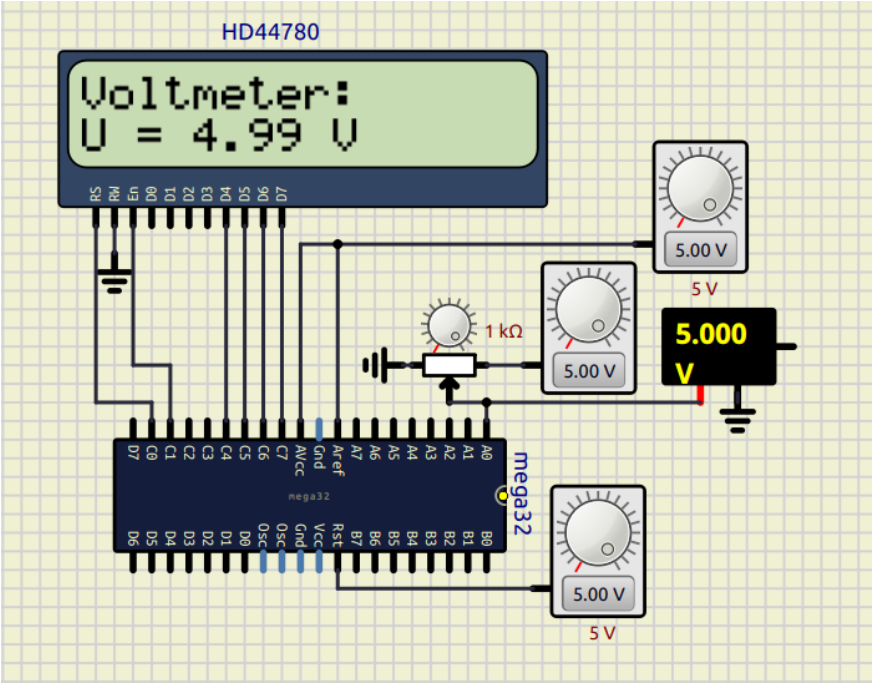
Ключовим етапом розробки стало низькорівневе конфігурування керуючих регістрів модуля АЦП. У регістрі мультиплексора `ADMUX` було встановлено біт `REFS0` для вибору напруги живлення `AVcc` в якості опорної, а біти вибору каналу залишено в нульовому стані для опитування входу `ADC0`. У регістрі керування та стану `ADCSRA` було встановлено біт `ADEN` для глобального увімкнення модуля, а також біти `ADPS2`, `ADPS1` та `ADPS0`, що конфігурують дільник тактової частоти (прескалер) на коефіцієнт 128. Це забезпечило оптимальну частоту тактування АЦП у межах рекомендованого діапазону (50-200 кГц) при головній частоті ядра 8 МГц.

Процес вимірювання реалізовано у головному нескінченному циклі шляхом встановлення біта запуску перетворення `ADSC` у регістрі `ADCSRA`. Програма очікує апаратного скидання цього біта, що сигналізує про завершення оцифрування, після чого зчитує 10-бітний результат безпосередньо з регістру даних `ADC`. Для перетворення отриманого «сирого» значення (від 0 до 1023) у реальні вольти було використано алгоритм оптимізованої цілочисельної арифметики: зчитане значення множилося на 500 та ділилося на 1023, дозволяючи уникнути ресурсомістких операцій з рухомою комою і одразу отримати напругу з точністю до сотих часток. Ціла та дробова частини виділялися за допомогою операцій цілочисельного ділення та залишку.

У результаті тестування системи було підтверджено повну працездатність моделі. При зміні положення повзунка потенціометра значення напруги на LCD-екрані миттєво оновлювалося і з високою точністю збігалося з показаннями еталонного віртуального вольтметра у всьому діапазоні вимірювань – від 0.00 В до максимальних 5.00 В.



Приклад з 5 вольтами:



Висновок. Під час виконання восьмої лабораторної роботи було успішно досліджено архітектуру, принцип дії та особливості програмування вбудованого 10-розрядного модуля АЦП мікроконтролера ATmega32. Було створено та налаштовано діючу апаратно-програмну модель цифрового вольтметра в середовищі SimulIDE. Практично закріплено навички конфігурування керуючих регістрів АЦП (`ADMUX`, `ADCSRA`), вибору джерела опорної напруги та узгодження частоти перетворення за допомогою вбудованого прескалера. Розроблено програмний алгоритм, який здійснює запуск АЦП, очікування прапорця завершення, зчитування результату перетворення та його математичну обробку без використання ресурсомістких чисел з рухомою комою для виведення реального значення напруги на LCD-дисплей. Отримані результати вимірювань підтвердили працездатність моделі та з високою точністю збіглися з показаннями еталонного віртуального вольтметра.

Контрольні запитання.

1. **Назвіть розрядність модуля АЦП AVR-мікроконтролерів?**
   До складу більшості моделей AVR-мікроконтролерів (зокрема ATmega32) входить модуль 10-розрядного аналого-цифрового перетворювача.
2. **Назвіть основні параметри АЦП.**
   До основних параметрів модуля АЦП належать: абсолютна похибка (зазвичай ± 2 одиниці молодшого розряду), роздільна здатність (10 біт) та діапазон вхідної напруги (від 0 до напруги живлення або опорної напруги).

3. **Назвіть основні джерела опорної напруги для АЦП.**

   В якості джерела опорної напруги (`Vref`) може виступати: зовнішня напруга, подана на вивід `AREF`; напруга живлення аналогової частини мікроконтролера (`AVcc`); або внутрішнє каліброване джерело опорної напруги (наприклад, 2.56 В для ATmega32).

4. **Назвіть режими роботи АЦП.**

   АЦП може працювати у двох основних режимах: режимі одиночного перетворення (Single Conversion mode), коли кожен цикл вимірювання запускається програмно, та режимі неперервного автоматичного перетворення (Free Running / Auto Triggering mode), де нове перетворення починається автоматично після завершення попереднього.

5. **У якому регістрі зберігається результат перетворення?**

   Результат перетворення зберігається у 16-бітному регістрі даних АЦП (ADC Data Register), який апаратно складається з двох 8-бітних регістрів: молодшого байта ADCL та старшого байта ADCH.

6. **Поясніть як саме відбувається перетворення вхідного сигналу у двійковий код?**

   Перетворення відбувається шляхом порівняння невідомої вхідної напруги з набором еталонних напруг, які генеруються внутрішнім цифро-аналоговим перетворювачем (ЦАП). Компаратор послідовно перевіряє кожен біт майбутнього результату, починаючи зі старшого (MSB) і закінчуючи молодшим (LSB), записуючи «1», якщо вхідний сигнал більший за поточний еталон, і «0», якщо менший.

7. **Яку роль у функціональній схемі модуля АЦП виконує ПВЗ?**

   ПВЗ (пристрій вибірки та зберігання, або Sample and Hold) виконує роль «засувки» для аналогового сигналу. Він миттєво фіксує рівень вхідної напруги на внутрішньому конденсаторі і підтримує його стабільним протягом усього часу перетворення, захищаючи процес оцифрування від похибок, спричинених коливаннями вхідного сигналу.

8. **Який принцип перетворення використовується в АЦП? Поясніть особливості цього принципу.**

   В АЦП мікроконтролерів AVR використовується принцип послідовного наближення. Головна особливість цього принципу полягає в алгоритмі дихотомії (поділу діапазону навпіл), що забезпечує оптимальний баланс між швидкістю та точністю: перетворення завжди займає фіксовану кількість тактів (зазвичай 13-14 тактів частоти АЦП для 10-бітного результату) незалежно

від величини вхідного сигналу.

9. **Який регістр відповідає за налаштування мультиплексора АЦП?**
   За налаштування вхідного мультиплексора (вибір аналогового каналу для вимірювання), а також за вибір джерела опорної напруги і форматування результату (вирівнювання) відповідає регістр `ADMUX` (ADC Multiplexer Selection Register).
10. **Наведіть та поясніть формати регістрів стану і керування АЦП.**
    Основним регістром керування є `ADCSRA` (ADC Control and Status Register A). Він містить такі керуючі біти: `ADEN` (глобальне увімкнення модуля АЦП), `ADSC` (запуск перетворення), `ADATE` (дозвіл автозапуску), `ADIF` (прапорець переривання по завершенню), `ADIE` (дозвіл переривання) та конфігураційні біти `ADPS2`, `ADPS1`, `ADPS0`, які встановлюють коефіцієнт дільника тактової частоти (прескалера) для забезпечення оптимальної швидкості роботи АЦП.
11. **Який розряд регістра `ADCSRA` відповідає за дозвіл роботи АЦП?**
    За глобальний дозвіл роботи модуля АЦП відповідає біт `ADEN` (ADC Enable), який знаходиться у регістрі керування та стану `ADCSRA`.
12. **Як обрати джерело опорної напруги?**
    Джерело опорної напруги обирається за допомогою двох керуючих бітів `REFS1` та `REFS0`, які розташовані у регістрі вибору мультиплексора `ADMUX`.
13. **Як вибрати режим роботи АЦП?**
    Режим роботи (одиночне перетворення або автоматичний перезапуск) вибирається шляхом встановлення або скидання біта `ADATE` (ADC Auto Trigger Enable) у регістрі `ADCSRA`, а конкретний метод запуску конфігурується у регістрі `SFIOR`.
14. **За якими перериваннями може відбуватися запуск АЦП?**
    Запуск АЦП може відбуватися за перериванням від зовнішнього запиту (наприклад, `INT0`), за перериваннями від таймерів/лічильників (переповнення, збіг порівняння, захоплення) або за перериванням від аналогового компаратора.
15. **Яку функцію виконує попередній дільник?**
    Попередній дільник (прескалер) використовується для масштабування високої системної тактової частоти мікроконтролера (наприклад, 8 МГц) до рекомендованого робочого діапазону АЦП (50-200 кГц), що необхідно для забезпечення максимальної роздільної здатності.
16. **Як формується тактовий сигнал АЦП?**
    Тактовий сигнал формується системним генератором мікроконтролера і

проходить через попередній дільник, коефіцієнт ділення якого (2, 4, 8, 16, 32, 64 або 128) задається керуючими бітами `ADPS2`, `ADPS1` та `ADPS0` у регістрі `ADCSRA`.

17. **Який регістр використовується для налаштування вхідного мультиплексора?**

    Для налаштування вхідного мультиплексора та вибору конкретного аналогового каналу вимірювання використовується регістр `ADMUX`.

18. **Поясніть часові діаграми роботи АЦП при першому одиночному перетворенні в режимі одиночного перетворення.**

    Перше одиночне перетворення вимагає найбільше часу – 25 тактів частоти АЦП. Додаткові такти витрачаються на ініціалізацію та стабілізацію аналогових схем модуля після його увімкнення.

19. **Поясніть часові діаграми роботи АЦП в режимі наступного одиночного перетворення.**

    Усі наступні (нормальні) одиночні перетворення відбуваються швидше і тривають рівно 13 тактів синхронізації АЦП.

20. **Поясніть часові діаграми роботи АЦП в режимі запуску за перериванням.**

    У цьому режимі перетворення ініціюється по позитивному фронту вибраного тригерного сигналу (переривання). Само перетворення триває стандартні 13.5 тактів (з урахуванням синхронізації тригера).

21. **Поясніть часові діаграми роботи АЦП в режимі автоматичного перезапуску для однополярного перетворення.**

    У режимі автоматичного перезапуску (Free Running) АЦП працює неперервно: як тільки завершується поточне перетворення (за 13 тактів), мікроконтролер автоматично, без програмного втручання, розпочинає наступне перетворення.

22. **Скільки тактів синхронізації АЦП витрачається на перетворення в режимах: першого одиночного перетворення та наступного одиночного перетворення?**

    На перше одиночне перетворення витрачається 25 тактів, а на кожне наступне одиночне перетворення – 13 тактів.

23. **Скільки однополярних входів має вхідний мультиплексор модуля АЦП?**

    Вхідний мультиплексор модуля АЦП (на прикладі ATmega32) має до 8 мультиплексованих каналів для підключення однополярних входів.

24. **Назвіть способи вирівнювання результату АЦП.**

Оскільки результат 10-бітний, а регістри 8-бітні, передбачено два способи вирівнювання результату: по правому краю (за замовчуванням) та по лівому краю. Це керується бітом `ADLAR` у регістрі `ADMUX`.

25. **Яке вирівнювання слід використовувати якщо досить точності 8-розрядного значення?**

Якщо достатньо 8-розрядної точності, зручно використовувати вирівнювання по лівому краю (встановити `ADLAR` в 1). Тоді результат можна зчитати за одну операцію лише зі старшого регістра `ADCH`, ігноруючи молодші 2 біти.

26. **Які особливості перемикання каналів при одиночному перетворенні?**

Безпечне перемикання каналів (запис у регістр `ADMUX`) слід здійснювати до встановлення біта запуску `ADSC` або відразу після того, як попереднє перетворення повністю завершиться.

27. **Які особливості перемикання каналів при перетворенні у режимі автоматичного перезапуску?**

У режимі автоперезапуску зміна каналу в регістрі `ADMUX` не вплине на перетворення, яке вже триває. Зміни наберуть чинності лише для наступного циклу АЦП.

28. **Що може виступати у ролі джерела опорної напруги?**

Роль джерела опорної напруги може виконувати: зовнішня напруга на виводі `AREF`, напруга живлення мікроконтролера `AVcc`, або внутрішнє стабільне джерело (наприклад, 2.56 В).

29. **Яким буде результат перетворення для каналів з однополярним входом?**

Для однополярного входу результат буде 10-бітним десятковим числом від 0 до 1023, прямо пропорційним відношенню вхідної напруги до опорної.

30. **Яким буде результат перетворення для каналів з диференціальним входом?**

Для диференціального входу результат визначається як різниця між додатним та від'ємним входами, помножена на підсилення, і подається у додатковому коді (зі знаком) у діапазоні від -512 до 511.

31. **Якою формулою визначається коефіцієнт передачі АЦП?**

Коефіцієнт передачі (або вага одного наймолодшого розряду) визначається формулою: $k = V_{ref}/1024$.

32. **Поясніть функцію перетворення АЦП при зміні однополярного сигналу.**

Функція перетворення є лінійною. Вона виражається як $ADC = \frac{V_{in} \cdot 1024}{V_{ref}}$, де мікроконтролер округлює отримане значення до найближчого цілого.

33. **Поясніть зв'язок між вхідним сигналом та вихідними кодами для однополярного режиму.**

    Якщо вхідний сигнал дорівнює 0 В, вихідний код – 0. Якщо вхідний сигнал лінійно зростає і наближається до значення V_{ref}, вихідний код досягає свого максимуму – 1023.

34. **Назвіть методи підвищення точності перетворення АЦП.**

    Основними методами є: фільтрація живлення `AVcc` через LC-ланцюжок, програмне усереднення серії зчитаних значень, а також переведення мікроконтролера у режим сну (ADC Noise Reduction) під час вимірювання для зменшення цифрових шумів.

35. **Чому дорівнює похибка зсуву та як її можна програмно врахувати?**

    Похибка зсуву – це відхилення коду від нуля, коли на вході фактично 0 Вольт. Програмно її враховують шляхом вимірювання нульової напруги (замикання каналу на землю) і віднімання отриманого значення-константи від усіх наступних результатів.

36. **Опишіть особливості програмування мовою С виведення на дисплей результату роботи модуля АЦП та цифрового вольтметра.**

    Особливістю є використання оптимізованої цілочисельної математики для запобігання ресурсомістким операціям з числами з плаваючою комою (float). Код АЦП множиться на масштабний коефіцієнт, після чого ціла та дробова частини виділяються через ділення та залишок.

37. **Як розраховується відносна похибка АЦП від квантування за рівнем?**

    Вона розраховується як відношення абсолютної похибки квантування (яка становить ± 0.5 LSB) до самого значення вхідного сигналу у відсотках.

38. **Опишіть схему моделювання модуля АЦП в пакеті SimulIDE.**

    Схема містить мікроконтролер ATmega32, підтягнуті до 5 В піни `AVcc` та `Aref`, потенціометр у ролі подільника напруги на вході PA0, та символьний LCD-дисплей для візуалізації розрахованих даних.

39. **В який режим програмується модуль АЦП при його моделюванні?**

    При моделюванні вольтметра модуль АЦП зазвичай програмується у режим одиночного перетворення, запуск якого циклічно здійснюється в основному нескінченному циклі програми.

40. **Чому дорівнює коефіцієнт передачі модуля АЦП при його моделюванні?**

Оскільки опорна напруга становить 5 В, а АЦП 10-розрядний, коефіцієнт передачі дорівнює $5/1024 \approx 0.00488$ В.

41. **В якому вигляді виводиться на індикацію результат моделювання модуля?**

    Результат виводиться у вигляді форматованого текстового рядка, де ціла частина (Вольти) відділена від дробової (соті частки Вольта) крапкою, наприклад «U = 2.50 V».

42. **За допомогою якої математичної операції визначається відповідна цифра при її виведенні на індикатор? Відповідь пояснити.**

    Для розрахунку відповідної цифри використовується цілочисельне ділення (наприклад, $/100$ для виділення цілих вольтів після масштабування) та операція отримання залишку ($\%100$ для виділення дробової частини). Це робиться для високої швидкодії виконання коду мікроконтролером.

43. **Який таймер/лічильник мікроконтролера використовується при виведенні на семисегментні індикатори?**

    Для динамічної індикації на семисегментних індикаторах зазвичай використовують 8-розрядний Таймер 0 або Таймер 2 для періодичного виклику переривань розгортки.

44. **Який сигнал використовується в якості тактового для таймера 2? Відповідь пояснити.**

    Таймер 2 може тактуватися асинхронно від зовнішнього кварцового резонатора (наприклад, годинникового на 32768 Гц), що підключається до виводів TOSC1/TOSC2, або ж синхронно від системної тактової частоти мікроконтролера через попередній дільник.

45. **Чим відрізняється виведення результату моделювання цифрового вольтметра від моделювання модуля АЦП?**

    При моделюванні суто модуля АЦП розглядається та індикується його «сирий» двійковий код (0-1023). Натомість моделювання цифрового вольтметра передбачає обов'язковий математичний перерахунок цього коду в реальні фізичні одиниці (Вольти) для зручності оператора.

Лістинг.

main.c

```c
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>

#define LCD_PORT PORTC
#define LCD_DDR DDRC
#define RS PC0
#define EN PC1

void LCD_Pulse(void) {
    LCD_PORT |= (1 << EN);
    _delay_us(1);
    LCD_PORT &= ~(1 << EN);
    _delay_us(50);
}

void LCD_Send(unsigned char value, unsigned char mode) {
    if (mode) LCD_PORT |= (1 << RS);
    else LCD_PORT &= ~(1 << RS);

    LCD_PORT = (LCD_PORT & 0x0F) | (value & 0xF0);
    LCD_Pulse();

    LCD_PORT = (LCD_PORT & 0x0F) | ((value << 4) & 0xF0);
    LCD_Pulse();
    _delay_ms(2);
}

void LCD_Init(void) {
    LCD_DDR = 0xFF;
    _delay_ms(20);

    LCD_PORT = (LCD_PORT & 0x0F) | 0x30;
    LCD_Pulse(); _delay_ms(5);
    LCD_Pulse(); _delay_us(150);
    LCD_Pulse(); _delay_us(150);

    LCD_PORT = (LCD_PORT & 0x0F) | 0x20;
    LCD_Pulse(); _delay_ms(2);

    LCD_Send(0x28, 0);
    LCD_Send(0x0C, 0);
    LCD_Send(0x01, 0);
    _delay_ms(2);
}

void LCD_String(char* str) {
    while (*str) {
        LCD_Send(*str++, 1);
    }
}

void LCD_SetCursor(unsigned char col, unsigned char row) {
    unsigned char address = (row == 0) ? (0x80 + col) : (0xC0 + col);
    LCD_Send(address, 0);
}

```

```c
void ADC_Init(void) {
    // Set AVCC as reference voltage and select ADC0 channel
    ADMUX = (1 << REFS0);

    // Enable ADC and set prescaler to 128 for optimal clock
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
}

uint16_t ADC_Read(void) {
    // Start analog to digital conversion
    ADCSRA |= (1 << ADSC);

    // Wait until conversion is fully complete
    while (ADCSRA & (1 << ADSC));

    // Return the 10-bit result directly from ADC register
    return ADC;
}

int main(void) {
    uint16_t adc_value;
    uint8_t volts;
    uint8_t millivolts;
    char buffer[16];

    LCD_Init();
    ADC_Init();

    LCD_SetCursor(0, 0);
    LCD_String("Voltmeter:");

    while (1) {
        adc_value = ADC_Read();

        // Convert 10-bit raw value to voltage avoiding float operations
        uint32_t temp = ((uint32_t)adc_value * 500) / 1023;

        volts = temp / 100;
        millivolts = temp % 100;

        LCD_SetCursor(0, 1);
        sprintf(buffer, "U = %d.%02d V   ", volts, millivolts);
        LCD_String(buffer);

        _delay_ms(200);
    }
    return 0;
}
```